

SentinelOps — architecture and build guide

[SentinelOps: implementation and architecture guide](#)

[SentinelOps](#)

[Start here](#)

[What you can do](#)

[Architecture](#)

[Quick start with Docker](#)

[Run natively for learning](#)

[Walk through one incident](#)

[Tests and evaluations](#)

[Optional OpenAI reasoning](#)

[Security model](#)

[Production deployment and remaining work](#)

[Recorded demonstration](#)

[Design decisions](#)

[Run SentinelOps on your own computer](#)

[1. Get and extract the source](#)

[2. Install and start Docker](#)

[3. Copy the local demo configuration](#)

[4. Build and start the app](#)

[5. Open and use the app](#)

[If the browser says “This site can’t be reached”](#)

[Stop and restart](#)

[Architecture: from an alert to a verified decision](#)

[1. Understand the two execution paths](#)

[2. Why separate an API and worker?](#)

[3. What each technology does](#)

[4. Workflow diagram](#)

[5. The data model and consistency rules](#)

[6. Trust boundaries](#)

[7. Local and cloud infrastructure are different](#)

[8. Design trade-offs](#)

[Build SentinelOps step by step: what to do and why](#)

[Step 1 — Define the outcome before choosing tools](#)

[Step 2 — Learn the request lifecycle](#)

[Step 3 — Create the repository structure](#)

[Step 4 — Define typed contracts](#)

Step 5 — Persist incidents and jobs before running work

Step 6 — Build read-only tool adapters

Step 7 — Retrieve relevant runbooks

Step 8 — Implement an explicit LangGraph workflow

Step 9 — Add optional model reasoning

Step 10 — Enforce policy before asking for approval

Step 11 — Bind human approval to an exact proposal

Step 12 — Execute with idempotency and safe recovery

Step 13 — Verify the operational outcome

Step 14 — Add an audit trail and telemetry

Step 15 — Evaluate before claiming quality

Step 16 — Package reproducibly

Step 17 — Prepare a real deployment

Project structure and how to navigate it

Read the code in this order

Follow one action through the repository

What belongs where when extending it

Security model and trust boundaries

1. Treat incident input and model output as untrusted

2. Authenticate the caller and authorize the action

3. Classify and constrain each tool

4. Bind approval to the exact action

5. Separate approval, scheduling, execution, and verification

6. Make audit history tamper evident

7. Keep live credentials behind the backend boundary

8. Deploy with least privilege and operational controls

Evaluating SentinelOps

1. Freeze the questions before measuring the answers

2. Run the real workflow

3. Understand exactly what each metric measures

4. Verify the full human approval lifecycle

5. Add stronger evaluation before customer use

Run and deploy SentinelOps

1. Understand what runs where

2. Install the local prerequisites

3. Choose explicit local configuration

4. Build and start the full stack

5. Verify the complete workflow

6. Run tests and measured evaluations

7. Run without Docker when learning each process

[8. Stop, inspect, or intentionally reset](#)

[Production architecture on Google Cloud](#)

[Production prerequisites, in order](#)

[Reviewable Cloud Run deployment steps](#)

[Operate and troubleshoot SentinelOps](#)

[Health and progress](#)

[Useful local commands](#)

[When something fails](#)

[Queue recovery and mutation boundaries](#)

[Approvals and audit](#)

[Backup and restore](#)

[Release and rollback](#)

[Alerts and capacity decisions for production](#)

[Validation record](#)

[Final application checks](#)

[Real local container checks](#)

[Evaluation interpretation](#)

[Recorded evidence](#)

[What remains unverified or intentionally unfinished](#)

[ADR 0001: use one explicit workflow orchestrator](#)

[ADR 0002: database-backed work requests](#)

[ADR 0003: approval authorizes an exact action](#)

[ADR 0004: reproducible simulation before live infrastructure](#)

SentinelOps: implementation and architecture guide

A working incident platform, explained from first principles. Includes every build step, its purpose, project navigation, safety, infrastructure, evaluation and validation.

Prepared 6 October 2026. The demonstrated operations use synthetic fixtures; production requirements are marked explicitly.

SentinelOps

An approval-gated incident investigation and simulated remediation platform. SentinelOps turns an incident into a traceable decision: gather observations, retrieve runbooks, form a hypothesis, propose a permitted action, obtain human approval, execute the exact approved action, verify recovery and generate a report.

The repository is a runnable production-oriented foundation and learning project. It does not connect to real enterprise infrastructure by default. Fixture tools simulate metrics, logs, deployments, database diagnostics and mutations. Optional OpenAI reasoning consumes the same evidence; it never grants execution authority.

Start here

If you are new to architecture, begin with [Architecture explained](#), then work through [Every step: what and why](#). These explain the building blocks, request lifecycle, project structure, infrastructure, safety and evaluation before cloud deployment.

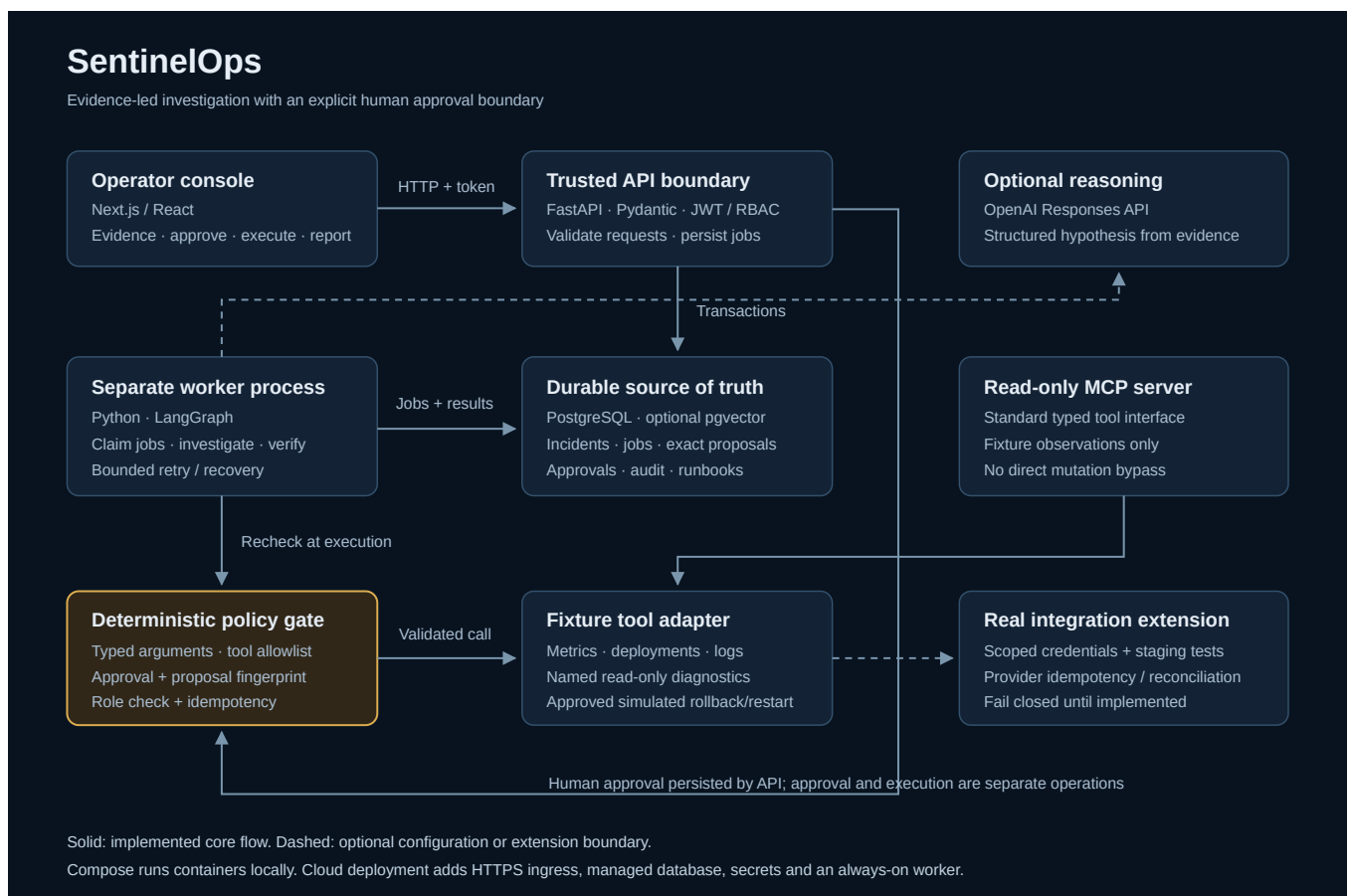
Question	Read
What does each component do?	Architecture and diagrams
How and why is every step implemented?	Detailed learning guide
Can I read all documentation in one place?	Offline HTML guide (included in the repository under docs/media/) · Printable PDF guide (included in the repository under docs/media/)
Where is each part of the code?	Project structure
What protects execution?	Security model
How are results measured?	Evaluation methodology
How do I deploy the containers?	Deployment guide
How do I recover failures and operate it?	Operations guide
What actually passed during this build?	Validation record
Why these design choices?	Architecture decision records

What you can do

- Create and investigate an incident from four reproducible scenarios.
- Inspect evidence, a structured hypothesis, curated runbooks and a step-by-step execution trace.
- Review a proposed rollback or restart, approve/reject it as an approver, then separately request execution.
- Verify simulated recovery and inspect the report and hash-chained audit history.
- Run a measured synthetic evaluation suite and inspect its report in the console.
- Inspect a typed read-only MCP interface without exposing a second mutation route.

The default investigation mode is deterministic and requires no OpenAI key. Database saturation and uncertain evidence produce manual follow-up instead of an unsupported automatic remediation. Confidence values are uncalibrated estimates, not probability guarantees.

Architecture



```

flowchart LR
    UI[Next.js console] --> API[FastAPI + JWT/RBAC]
    API --> DB[(PostgreSQL: incident / job / approval / audit)]
    WORKER[Worker + LangGraph] --> DB
    WORKER --> RAG[Runbook retrieval]
    WORKER --> POLICY[Typed deterministic policy]
    WORKER -. optional .-> LLM[OpenAI Responses API]
    POLICY --> TOOLS[Simulated tool adapters]
    HUMAN[Human approver] --> UI
    MCP[Read-only MCP] --> TOOLS

```

The API and worker share code and database records, not process memory. PostgreSQL owns durable state. SQLite is a single-worker native-development fallback. Runbook retrieval uses a keyword baseline; pgvector schema support is an extension point for a proper embedding ingestion/search pipeline. Redis and the telemetry collector are optional infrastructure profiles, not prerequisites for the current database-backed queue.

Quick start with Docker

Requirements: Docker Engine/Desktop with Compose v2, enough memory for a Next.js build, and access to npm/Python/container registries. No cloud account or model key is required.

Download/extract the project onto your own computer and open a terminal in the `SentinelOps` folder containing `compose.yaml`. Docker Desktop must be running on that computer. These steps are also available with Windows and macOS/Linux examples in [Local setup and connection troubleshooting](#).

```

cp .env.example .env
docker compose up --build -d --wait

```

Open the console at **`http://localhost:3000`** and interactive API documentation at **`http://localhost:8000/docs`**. `localhost` means the computer running your browser: these addresses work when Docker is running on that same computer. Starting containers in a remote Codex cloud workspace does not start the app on your laptop or make cloud loopback addresses reachable there.

The browser talks directly to the API, so `NEXT_PUBLIC_API_URL` must be reachable from the browser. This setting is embedded into the frontend at build time. If you change ports in `.env`, update `NEXT_PUBLIC_API_URL` and `CORS_ORIGINS` to match and rebuild the frontend. Remote access needs a separately configured hosting or forwarding mechanism; no browser-accessible preview URL is provided by the source repository itself.

The demo identities are:

Identity	Password	Permission
operator	sentinel-demo	Create and investigate incidents
approver	sentinel-demo	Investigate, approve/reject and execute permitted remediation

These are public local-demo credentials. The Compose demo binds web/API ports to loopback and does not publish PostgreSQL or Redis ports. Disable demo login and replace signing configuration before any shared deployment. Native configuration defaults to demo login disabled.

Compose starts PostgreSQL, runs migrations and seeds runbooks, then starts the API, worker and web app. The optional cache and collector can be started explicitly:

```
docker compose --profile cache --profile telemetry up -d
```

Starting those profiles does not automatically add caching or OTLP exports. Native `OTEL_CONSOLE_EXPORT=true` enables local console spans; production collector export and cross-process propagation remain extension work. See the deployment guide for the current integration boundaries.

Useful operations:

```
docker compose ps
docker compose logs -f api worker
docker compose down
```

Optional: if Python is installed on your computer, run `python infra/smoke_test.py` while the app is running to check the incident workflow. Starting the app with Docker does not require Python installed on your computer.

Ordinary `down` retains the named database volume. Do not remove volumes unless you intend to discard local incident history.

Run natively for learning

Requirements: Python 3.12+, Node.js 22+, npm. Use SQLite and one worker for this path. Run commands from the repository root unless a `cd` is shown.


```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --require-hashes -r apps/api/requirements-dev.lock
python -m pip install --no-deps -e apps/api
cp .env.example apps/api/.env
cd apps/api
python -m sentinelops.migrate
python -m sentinelops.seed
uvicorn sentinelops.main:app --host 127.0.0.1 --port 8000
```

In a second terminal, activate the same environment and start the worker from `apps/api`:

```
source .venv/bin/activate
cd apps/api
python -m sentinelops.worker
```

In a third terminal, start the frontend:

```
cd apps/web
npm ci
npm run dev
```

The API and worker must use the same `DATABASE_URL`; two different SQLite files would create two isolated systems. Both read `apps/api/.env` when started from that directory. On Windows, use the corresponding virtual-environment activation command and an appropriate shell for these commands.

Walk through one incident

1. Sign in as **operator** and create the checkout deployment-regression scenario.
2. Click **Investigate**. The API queues durable work; the worker gathers fixture observations and produces the evidence and proposal.
3. Inspect the hypothesis and the rollback proposal. Investigation stops at the policy gate and performs no mutation.
4. Sign in as **approver**, reopen the incident, enter a reason and approve the exact proposal.
5. Click **Execute**. The worker rechecks authorization and changes simulated deployment state.
6. Review verification, the final report, workflow trace and audit events.

Try database saturation or an unknown scenario next. The useful outcome may be escalation rather than an automatic action. A safe incident responder needs to recognize that difference.

Tests and evaluations

With the Python environment active, from the root:

```
python -m pytest tests apps/api/tests -q
cd apps/api
python -m sentinelops.evaluate --output ../../evals/results/latest.json
```

Then open the Evaluations view. The report is generated from execution, not seeded with invented scores. The fixture corpus contains explicit telemetry and expected outcomes in separate fields; expected answers are not passed to the workflow.

Frontend checks:

```
cd apps/web
npm run typecheck
npm run build
```

Backend tests check policy and lifecycle behavior, including role denial, approval binding, typed tool arguments and idempotency. Evaluation metrics are synthetic fixture baseline results. They do not establish real-model accuracy, semantic retrieval quality, cloud latency or production remediation reliability. See [evaluation.md](#) and the [validation record](#) for the exact scope.

Optional OpenAI reasoning

For Compose, edit the root `.env` ; for native processes, edit `apps/api/.env` :

```
LLM_MODE=openai
OPENAI_API_KEY=your-server-side-key
OPENAI_MODEL=your-supported-model-id
```

Restart the API and worker after changing backend settings. For Compose, use `docker compose up -d --force-recreate api worker` . The default example model is a configurable starting point, not a guarantee of availability for your account. OpenAI mode uses structured hypothesis output from the Responses API. Production tool adapters remain disabled.

The supplied validation was run without provider credentials. Do not claim a live-agent score or token cost until you run a separately labeled provider-backed suite and record actual usage.

Security model

Read tools are allowlisted and typed. The SQL diagnostic tool accepts named adapter-owned templates, not arbitrary SQL. `rollback_deployment` and `restart_service` require an approver and an exact matching persisted approval. `write_sql` , `delete_data` and unknown tools are blocked. API checks and worker checks enforce the boundary independently of the UI and model.

Audit records are append-only through the application ORM and hash-chained for integrity checking. This does not make the database immutable against administrators or direct SQL. Stronger retention requires database grants and external immutable storage/anchoring.

JWT signature, issuer, audience and expiry are validated. Demo login is intentionally local-only. A configurable verification key provides a token validation boundary, but enterprise OIDC login, JWKS rotation, organization mapping and browser session hardening still require deployment-specific work.

Production deployment and remaining work

The GCP deployment guide maps the containers to Cloud Run, Cloud SQL, Secret Manager and an always-on worker. Cloud manifests are templates to review and configure, not evidence of a deployed cloud environment. No cloud resources were provisioned by this build. Terraform is a documented next step rather than a misleading empty infrastructure promise.

Before serving real customers, complete and validate:

- Enterprise identity, service/environment-scoped RBAC, tenant isolation where needed, and protected browser sessions.
- Authorized live adapters with scoped credentials, timeouts, retries, provider idempotency and ambiguous-outcome reconciliation.
- Permission-aware runbook ingestion, versioned embeddings, semantic retrieval and retrieval-specific evals if pgvector is enabled.
- Approval expiry/change windows, separation of duties, external audit retention and secret rotation.
- Production database migrations, backup restore drills, worker/concurrency tests, load tests and capacity limits.
- Exported telemetry, dashboards, alerting, SLOs and an on-call runbook.
- Staging failure exercises, deployment rollback, dependency/security review and measured live-model latency/cost.

Each item has a reason and a verification step in the [learning guide](#) and [operations guide](#). This separation lets you demonstrate real engineering work without presenting simulation as production evidence.

Recorded demonstration

These artifacts were captured from the running fixture application, rather than drawn as mockups:

- Incident creation, approval, execution and resolution video (included in the repository under docs/media/)

- Investigation trace and supporting evidence (included in the repository under docs/media/)
- Measured evaluation dashboard (included in the repository under docs/media/)
- Resolved incident report (included in the repository under docs/media/)
- Mobile console (included in the repository under docs/media/)

The recording and screenshots use simulated telemetry. The complete PostgreSQL container path was separately verified; see the validation record.

Design decisions

- Use one explicit workflow orchestrator
- Persist work requests in the database
- Approve exact actions
- Build reproducible fixtures before live integrations

The project deliberately avoids combining multiple agent orchestration frameworks. The essential engineering decisions are who owns state, who may authorize a mutation, what survives a crash, and how an outcome is verified.

Run SentinelOps on your own computer

The source project was built in a remote Codex cloud workspace. A browser address beginning with `localhost` points to the computer running that browser. Containers running in the cloud workspace therefore cannot be opened through your laptop's `localhost` address.

To use `http://localhost:3000`, put the source project and run Docker on the same computer as your browser. This guide uses the repository's default ports. No model key or cloud account is needed for the fixture demo.

1. Get and extract the source

Open [the SentinelOps repository](#), select **Code** → **Download ZIP**, and extract `SentinelOps-main.zip`. Open a terminal in the extracted `SentinelOps-main` folder containing `compose.yaml`, `.env.example`, `apps`, `docs`, and `infra`.

If using the standalone `SentinelOps-source.zip` archive instead, extract it and open a terminal in its `SentinelOps` folder. With Git installed, you can also download the project using:

```
git clone https://github.com/ckiran24/SentinelOps.git
cd SentinelOps
```

Why: Compose reads `compose.yaml` from the current directory. Running commands in an unrelated folder will produce a configuration-file error rather than start this project.

2. Install and start Docker

Install Docker Desktop for Windows or macOS using Docker's instructions for your operating system. Start Docker Desktop and wait until the engine is running. Linux users can use Docker Engine with the Compose v2 plugin.

Verify in the terminal:

```
docker version
docker compose version
```

`docker version` should show both a Client and a Server. A missing Server or daemon connection error means the Docker engine is not running or is inaccessible. Windows installations may require WSL 2 and a restart; follow Docker Desktop's installation prompts.

Why: Docker runs the web app, API, worker, and PostgreSQL together. You do not need to install Node.js, Python, or PostgreSQL separately for this route.

3. Copy the local demo configuration

In Windows PowerShell:

```
Copy-Item .env.example .env
```

In macOS/Linux Terminal:

```
cp .env.example .env
```

If `.env` already exists, inspect it before copying over it. The example contains public demo credentials and local loopback bindings. Keep the default values for this first run:

```
WEB_PORT=3000
API_PORT=8000
NEXT_PUBLIC_API_URL=http://localhost:8000
CORS_ORIGINS=http://localhost:3000
```

Why: These settings must agree about where the browser can find the API and which web origin the API accepts. `NEXT_PUBLIC_API_URL` is included in the frontend at build time.

4. Build and start the app

Run this from the folder containing `compose.yaml` :

```
docker compose up --build -d --wait
docker compose ps -a
```

The initial build downloads dependencies and takes longer than later starts. Wait for it to finish successfully before opening the browser. PostgreSQL, API, worker, and web should be running and healthy. The one-time `migrate` service should show `Exited (0)` ; that is a successful migration, not a broken server.

Why: Compose builds the application images, starts PostgreSQL, creates its schema, seeds runbooks, and then starts dependent services. The `--wait` option waits for service readiness; `-d` leaves them running in the background.

5. Open and use the app

Open these addresses in a browser on the computer running Docker:

- Console: `http://localhost:3000`
- API documentation: `http://localhost:8000/docs`
- API readiness: `http://localhost:8000/health/ready`

For the demo, sign in as `operator` with password `sentinel-demo`. Create and investigate an incident. Sign in as `approver` with the same password to review a proposal and approve it. Execution is a separate action after approval.

The default fixture mode simulates infrastructure observations and remediation. It does not change real infrastructure.

If the browser says “This site can’t be reached”

1. Confirm the source is on this computer and Docker Desktop is running here. Cloud containers do not supply your computer’s localhost server.
2. Run `docker compose ps -a` from the project folder. The `web` service should publish `127.0.0.1:3000->3000/tcp`; the API should publish `127.0.0.1:8000->8000/tcp` with default settings.
3. If services are missing, stopped, or unhealthy, inspect the errors:

```
docker compose logs --tail=100 web api migrate postgres
```

4. If `.env` uses another `WEB_PORT`, open that port instead. To resolve a port conflict, change all corresponding values together. For example:

```
WEB_PORT=3100
API_PORT=8100
NEXT_PUBLIC_API_URL=http://localhost:8100
CORS_ORIGINS=http://localhost:3100
```

Then run `docker compose up --build -d --wait` again and open `http://localhost:3100`. These are an alternative pair of ports, not the defaults.

5. Use `http://`, since this local demo does not configure HTTPS. If localhost name resolution is suspect, compare the page at `http://127.0.0.1:3000`; use the configured localhost console origin for the full authenticated demo.

If the console loads but shows API connection errors, open the readiness URL and check the API logs. Confirm `NEXT_PUBLIC_API_URL` and `CORS_ORIGINS` match your chosen ports, then rebuild after correcting them.

Stop and restart

```
docker compose down
docker compose up -d --wait
```

Ordinary `down` keeps the named PostgreSQL volume, so incident history persists. Removing volumes discards that history; it is not needed for connection troubleshooting.

Architecture: from an alert to a verified decision

Architecture means deciding which component owns each responsibility, how components communicate, and what happens when one fails. Choosing a framework is only one part of that decision. SentinelOps separates human interaction, trusted policy, long-running work, external observations, and durable records.

1. Understand the two execution paths

Investigation is read-only. An operator creates an incident. A worker classifies it, retrieves runbooks, reads simulated service metrics/deployments/logs, builds a hypothesis and evidence, and proposes an action. Policy evaluates that action. The result is either an approval request or a case requiring more investigation.

Remediation crosses a trust boundary. An approver reviews a specific proposal and records a reason. Approval does not automatically execute it. The approver then requests execution. The worker revalidates the exact approved proposal and records the outcome. Verification checks service behavior after the simulated change; a successful tool return alone cannot resolve an incident.

```
flowchart TB
    U[Operator / Approver] --> WEB[Next.js / React console]
    WEB -->|HTTPS + bearer token| API[FastAPI: validation, RBAC, incident API]
    API -->|transactions| DB[(PostgreSQL: incidents, jobs, approvals, audit)]
    WORKER[Python worker + LangGraph] -->|claim durable jobs| DB
    WORKER --> POLICY[Typed tool registry + deterministic policy]
    WORKER --> RAG[Runbook retrieval]
    RAG --> CURATED[Curated runbooks in code]
    RAG -. future database/vector retrieval .-> DB
    WORKER --> LLM[Optional OpenAI Responses API]
    POLICY --> FIXTURE[Fixture tool adapter: simulated systems]
    MCP[MCP server: read-only tool interface] --> FIXTURE
    WORKER -. deployment extension .-> REAL[Real metrics / logs / deployment adapters]
    WORKER -. optional best-effort cache .-> REDIS[(Redis)]
    API -. telemetry .-> OTEL[OpenTelemetry collector]
    WORKER -. telemetry .-> OTEL
```

Dashed connections represent optional components or an extension boundary; they are not evidence that a real enterprise integration is configured. PostgreSQL is the shared operational source of truth. Runbook browsing is backed by seeded database rows; investigation retrieval currently uses the matching curated definitions in code. SQLite is a convenient single-worker local fallback. Redis must never be the sole holder of an approval or an execution request.

2. Why separate an API and worker?

An HTTP API should answer promptly. Investigation may spend seconds waiting for an external service and may later wait hours for a person. Keeping that work inside the incoming request makes browser disconnects, reverse-proxy timeouts and process restarts affect the workflow.

The API validates the request and persists a job. A separate worker claims it and saves results. That gives us a place to add bounded retries, job leases and execution-specific recovery. The browser polls the incident detail while work is active. A websocket would change delivery to the browser; it would not replace durable state.

The initial application uses a database job table rather than adding a second queue as another source of truth. PostgreSQL supports transactional job claims. A larger deployment can use a managed queue, but publication and consumption must account for duplicate delivery and transaction boundaries, usually with an outbox. Redis is useful for expendable cache entries, not as the authoritative job ledger.

3. What each technology does

Component	Job in this project	Why it exists
Next.js + React	Operator console and interactive incident details	People need to inspect evidence and make explicit decisions.
FastAPI	HTTP endpoints and dependency-based authentication	Provides typed API boundaries and generated OpenAPI documentation.
Pydantic	Incident, proposal and tool argument validation	Rejects malformed inputs before application logic uses them.
SQLAlchemy	Database access and transactions	Keeps application models coherent across PostgreSQL and the local SQLite fallback.
PostgreSQL	Operational records and durable jobs	Preserves state and coordinates concurrent processes.
pgvector	Optional runbook embedding storage/search	Supports semantic retrieval when embeddings and an index are configured. The implemented retrieval is keyword ranking over curated in-code documents; ingestion, embedding generation, vector indexing and semantic search are not implemented.

Component	Job in this project	Why it exists
LangGraph	Explicit investigation steps and conditional transitions	Makes the workflow inspectable and avoids an unbounded agent loop. Lifecycle routing is handled by application code.
OpenAI Responses API	Optional evidence-grounded hypothesis generation	Provides reasoning assistance; the model has no authority to approve or execute tools.
MCP	Typed external tool interface	Lets integrations be exposed consistently, while application policy remains the enforcement point.
Redis	Optional short-lived cache	Can reduce repeated reads; losing cache data must not lose incident progress.
OpenTelemetry	Workflow span hooks and an optional receiver scaffold	Stored workflow traces work locally; OTEL_CONSOLE_EXPORT=true enables local console spans. OTLP exports and cross-process request/job trace propagation are not wired in this version.
Docker Compose	Local multi-process environment	Makes the API, worker, web app and PostgreSQL run with known dependencies.
GitHub Actions	Repeatable checks on changes	Prevents known test/build regressions from reaching the main branch.

4. Workflow diagram

```

stateDiagram-v2
    [*] --> new
    new --> investigating: enqueue investigation
    investigating --> awaiting_approval: supported allowed proposal
    investigating --> needs_attention: missing / uncertain evidence
    investigating --> failed: unrecoverable investigation error
    awaiting_approval --> approved: approver records reason
    awaiting_approval --> rejected: approver declines
    approved --> executing: explicit execution request
    executing --> resolved: post-action verification passes
    executing --> needs_attention: verification fails / result ambiguous
    executing --> failed: definite failure
    resolved --> [*]
    rejected --> [*]

```

An approval is a database record, not a message to the model. Worker execution is a separate operation. The compiled graph currently has a bounded sequential investigation path that ends at policy. The worker maps its result to the lifecycle states above. The application persists user-visible state at job boundaries; do not confuse this with a fully configured LangGraph checkpointing that can resume inside any node after a crash.

5. The data model and consistency rules

- **Incident:** the alert, service, severity and current lifecycle status.
- **Investigation result:** hypothesis, evidence and trace entries.
- **Proposal:** exact tool name and arguments plus a canonical fingerprint.
- **Approval:** approver identity, reason and the proposal fingerprint it authorizes.
- **Job:** durable request, claim/lease metadata and terminal outcome.
- **Audit event:** actor, event, timestamp, payload and integrity-chain hash.
- **Runbook:** service guidance and optional embedding for retrieval.

The important invariant is: an execution refers to an approval for the same proposal content, in the same incident, with the required role. Updating arguments after approval invalidates the approval. Another invariant is that repeating an execution request must not apply the same action twice. The application uses idempotency records and persisted state to enforce this for its simulated adapters. Real integrations must support their own idempotency or reconciliation too.

SQL transactions group changes that must agree. For example, approving a proposal and writing its audit event should commit together. Locks coordinate concurrent requests. Hashing an action does not replace those database controls.

6. Trust boundaries

1. **Browser to API:** the browser is untrusted. API validation and authorization apply even if someone bypasses the interface.
2. **Model output to policy:** model output is untrusted data. It must match a typed schema and a tool allowlist. Retrieved runbooks and logs are untrusted too; text inside them cannot grant permissions.
3. **Policy to adapter:** every mutation requires a valid approval at execution time. High-risk tool credentials should be isolated from read-only tools.
4. **Application to database:** an ordinary application user must not update/delete audit events. A database administrator remains powerful; external immutable retention is required for stronger guarantees.
5. **Service to provider:** credentials stay server-side; production egress should restrict destinations and tool scopes.

The fixture adapter deliberately changes simulated state only. The project does not connect to your production deployment or database, and an OpenAI key does not enable live remediation.

7. Local and cloud infrastructure are different

```
flowchart LR
    B[Browser] --> TLS[HTTPS ingress / load balancer]
    TLS --> W[Next.js container]
    TLS --> A[FastAPI service]
    A --> PG[(Managed PostgreSQL + pgvector)]
    WK[Always-on worker service] --> PG
    WK --> OAI[OpenAI HTTPS API]
    A --> SM[Secret manager]
    WK --> SM
    A --> LOG[Logs / traces / metrics]
    WK --> LOG
    WK -. optional .-> C[(Managed Redis)]
```

Compose runs these processes on one developer machine. Cloud Run or ECS runs the containers under a managed scheduler. Managed PostgreSQL supplies backups, high availability options and database operations; container orchestration alone does not. The polling worker requires CPU while idle, a minimum running instance and a clear concurrency model. A request-only scale-to-zero service is unsuitable for the current polling worker. See [deployment.md](#) for the concrete GCP path and [operations.md](#) for recovery procedures.

8. Design trade-offs

One application database first. This simplifies consistency but makes the database part of both request and worker availability. Monitor connections and queue depth. Split analytical history or vector search only when workload evidence justifies it.

Deterministic policy around a probabilistic model. The model can assist diagnosis. Approval requirements, permission checks and forbidden operations must be deterministic code that remains effective when the model is wrong.

A fixture environment before live adapters. This makes the workflow reproducible and safe to demonstrate. It does not prove that a real rollout system, metrics source or database behaves correctly. Contract tests and staged failure injection are required for each integration.

Explicit state over framework accumulation. LangGraph is the single workflow orchestrator. Responses API is the model interface. MCP is a tool protocol. Adding another orchestration SDK would add competing state and recovery semantics without solving a current requirement.

Build SentinelOps step by step: what to do and why

Read this alongside the code. The order follows the dependencies of the system: understand the problem, define contracts, persist state, investigate, apply policy, involve a human, execute, verify, then operate and evaluate it.

Step 1 — Define the outcome before choosing tools

The outcome is a trustworthy incident decision with supporting evidence, and a verified action when approved. A fluent chatbot response is not enough. For the example incident, the starting observation is “checkout API errors rose after a release.” A useful ending is either a verified simulated rollback or a clear escalation explaining which evidence is missing.

Write down the actor and permitted action: an operator may submit and investigate; an approver may authorize and execute a high-risk remediation. The first implementation has a single organization and two demo roles. Tenant isolation, enterprise identity lifecycle and service-specific entitlements are production extensions.

Why: clear outcomes determine which endpoints, database records and tests matter. They also make it possible to measure correctness rather than judging the agent’s writing style.

Verify: run the checkout scenario end to end and inspect evidence, approval, execution and verification. A diagnosis without the approval boundary is incomplete.

Step 2 — Learn the request lifecycle

React renders what the user sees. It calls FastAPI over HTTP with a bearer token. FastAPI validates the body using Pydantic, checks the user’s role, opens a database transaction and returns a typed result. The worker handles queued work independently. The browser refreshes the incident while its status is investigating or executing.

The browser cannot be trusted to hide dangerous operations. A disabled button is a usability feature; the API role check is the security control. Generated `/docs` OpenAPI documentation makes the contract inspectable and lets you try the same requests without the UI.

Why: a clear request boundary keeps presentation, application rules and persistence separate. Each can change without giving the browser or model more authority.

Verify: call an approval endpoint with the operator token. It must be denied even if you write the HTTP request manually.

Step 3 — Create the repository structure

Keep frontend code in `apps/web`, backend code in `apps/api/sentinelops`, system tests in `tests`, fixtures and measured outputs in `evals`, infrastructure in `infra`, and explanations in `docs`. This is a monorepo: one repository containing several cooperating applications.

The API and worker import the same Python domain logic. They run as separate processes, so shared variables in Python memory do not coordinate them. They share database records instead.

Environment files provide configuration without hard-coding provider keys or production addresses.

Why: ownership and dependency boundaries are easier to understand when the filesystem reflects them. A monorepo also makes a change to API and UI contract reviewable in one change.

Verify: follow [project-structure.md](#) to the entrypoints and trace one endpoint to its workflow and database writes.

Step 4 — Define typed contracts

Incident input has a title, service, severity and description. Proposal arguments have a tool-specific schema. Unknown tools and unexpected fields should be rejected. Structured model output describes a hypothesis; it is not an arbitrary script to execute.

Use closed choices where the domain is bounded: severity, lifecycle states, root-cause categories and tool names. Use free text for human descriptions and supporting evidence. Avoid accepting raw SQL or arbitrary shell commands merely because that gives the agent flexibility.

Why: typed boundaries turn malformed output into a visible error rather than a surprising side effect. They also create stable interfaces for the frontend and tests.

Verify: try an unknown tool, invalid service identifier, unexpected argument and raw SQL mutation in policy tests. Each should fail before an adapter call.

Step 5 — Persist incidents and jobs before running work

An incident is a record, not a transient prompt. Persist its status and investigation result. An investigation request creates a durable job. A worker claims available work, performs bounded steps and saves the outcome. Lease/recovery rules decide what happens if it crashes.

Use PostgreSQL in the shared multi-process environment. SQLite is a simple local option for learning with one worker; it does not validate PostgreSQL row-locking behavior or production throughput. Database migrations version schema changes; initial schema creation alone is not a safe upgrade strategy.

Why: incidents can last longer than an HTTP request or process lifetime. Durability makes waiting for approval, auditing and recovery possible.

Verify: create an incident, stop the worker before processing, restart it and see that the queued investigation still runs. Inspect the job row and incident status if it fails.

Step 6 — Build read-only tool adapters

The agent needs observations: deployment history, metrics, logs, database diagnostics and runbooks. An adapter translates a typed tool call into a provider request. In this version, fixture adapters return reproducible system observations and simulate mutations. The MCP server exposes a read-only interface; privileged mutation remains behind application approval policy.

For a real integration, define its authentication, timeout, rate limit, data redaction, error shape and idempotency contract. Use read-only credentials for read tools. A tool labeled “read-only” but backed by an administrator database account is not properly isolated.

Why: integration behavior is a domain boundary. Keeping it out of the graph nodes makes failures testable and providers replaceable.

Verify: invoke a read tool directly in an isolated test, confirm its schema and check that it cannot mutate the fixture. Never use the MCP server as an unauthenticated route around the API.

Step 7 — Retrieve relevant runbooks

RAG means retrieval-augmented generation: fetch relevant documents, then provide them as context. A runbook helps the hypothesis and recommendation refer to known operational procedures. The local baseline ranks curated in-code runbooks using keywords without requiring paid embeddings; the database has matching seeded records for browsing. PostgreSQL with pgvector provides an extension path for semantic retrieval.

A production ingestion pipeline needs document identifiers, chunk boundaries, source permissions, document versions, embedding model/version, reindex behavior and deletion propagation. An embedding column alone is not a complete RAG system. Never send unrelated secrets or unauthorized documents to the model.

Why: retrieval makes recommendations grounded in the organization's procedures and reduces reliance on model memory. It also creates a measurable component: did the expected document appear among the retrieved results?

Verify: run retrieval evals and inspect returned runbook IDs. Keep the measured local baseline distinct from a future semantic retrieval benchmark.

Step 8 — Implement an explicit LangGraph workflow

Use named steps for triage, retrieval, observation collection, hypothesis, supporting evidence and proposal. State passes between steps. The bounded graph ends at its policy gate. Application code routes a supported proposal to approval and routes insufficient evidence to human attention.

The graph describes how investigation proceeds. Durable application records describe where the incident stands. Do not assume choosing LangGraph automatically configures persistent checkpoints, distributed locks or retries. The implementation persists results at job boundaries; a production checkpointing strategy must specify how intermediate steps resume and how duplicate side effects are prevented.

Why: explicit steps make behavior inspectable and bounded. You can see which observation produced a recommendation and test specific transitions.

Verify: inspect the trace view and compare it to the workflow diagram. The trace should show observations and decisions, not hidden model chain-of-thought.

Step 9 — Add optional model reasoning

In offline demo mode, diagnosis is deterministic over fixture observations. With the optional OpenAI configuration, the server can ask the Responses API for a structured hypothesis grounded in evidence. Configure the model identifier yourself and keep the API key server-side.

Treat model output as untrusted. Validate categories and confidence fields. The model does not issue an approval, choose a user's role or supply executable SQL. Provider failure must lead to a documented fallback or visible failure; it must never silently permit a mutation.

Why: model reasoning can help with ambiguity, while deterministic code protects invariants that must hold regardless of the model's answer.

Verify: test without a key first. Then test model mode in staging with recorded token usage and provider errors. This workspace has no configured OpenAI credential; live model behavior is not part of the recorded local validation.

Step 10 — Enforce policy before asking for approval

Maintain a tool registry with typed arguments and risk levels. Read tools are permitted with their scopes. Restart and rollback require human approval. SQL writes, deletion and unknown tools are blocked. The policy function is ordinary application code, independent of the model prompt.

Prefer named database diagnostic queries over model-written SQL. With named templates, the application controls the SQL and allowed parameters. Production also needs a database account limited to reading the allowed tables; application validation and database privileges work together.

Why: an agent can make a plausible but unsafe recommendation. The policy gate prevents that recommendation from becoming a privileged tool call.

Verify: adversarial policy cases should include unknown tools, destructive operations, raw SQL, invalid arguments, absent approvals and insufficient roles.

Step 11 — Bind human approval to an exact proposal

A person must be able to see what will happen: tool, service, target version or restart scope, risk and evidence. Approval records who authorized it and why. A canonical fingerprint binds the decision to the exact action and arguments. Execute is a separate explicit operation.

An approval is not permission for all future actions in that incident. If the proposal changes, request another approval. A production extension should add expiry, environment/service entitlements, separation of duties and policy-defined change windows.

Why: “approve the incident” is too broad. The meaningful unit of approval is a specific remediation in a specific context.

Verify: mutate a proposal after approval in a test and try to execute. It must fail. Also test duplicate requests, approval races and a wrong-role caller.

Step 12 — Execute with idempotency and safe recovery

The execution request includes an idempotency key. The API and worker verify current incident state, approval and fingerprint. The simulated adapter records its outcome so repeated requests do not apply a second mutation.

In real infrastructure, an adapter may succeed remotely but fail to return its response. Automatically retrying can duplicate the mutation. Persist intent, pass a provider-supported idempotency key, and reconcile remote state after uncertainty. If you cannot prove whether the action happened, mark the incident for attention rather than pretending it failed safely.

Why: distributed systems can produce duplicate delivery and ambiguous outcomes. Exactly-once external execution is not guaranteed by a single application flag.

Verify: execute twice using the same key and inspect the mutation ledger and audit trail. Add a provider-specific “response lost after success” integration test before enabling real actions.

Step 13 — Verify the operational outcome

After a rollback/restart simulation, query the service again. Compare post-action observations with the incident's success criteria. Verification should determine whether error rate or resource pressure improved. An adapter returning “OK” proves that its request succeeded, not that the incident was solved.

Write a report with initial observations, evidence, hypothesis, action, approver, verification and remaining uncertainty. Keep confidential logs out of exports according to the organization's retention and redaction policy.

Why: verified outcomes are the business value. A system that performs actions without measuring results can make an incident worse while reporting success.

Verify: run the successful scenarios and failure/unknown scenarios; only verified cases may resolve. The fixture verification is a simulation, not evidence of production recovery.

Step 14 — Add an audit trail and telemetry

Audit events record important decisions. Hash chaining makes edits detectable by an integrity verifier, but it is not absolute immutability: a privileged database user can rewrite events and recompute a chain. Production needs restricted database permissions and external append-only/WORM retention or externally anchored hashes.

Operational telemetry answers a different question: how is the application running? The current workflow persists readable trace entries; `OTEL_CONSOLE_EXPORT=true` enables SDK console spans. The optional collector is a receiver scaffold; OTLP export and cross-process context propagation are not configured. Production structured logs and OpenTelemetry spans should carry incident/job identifiers, tool names, durations and outcomes, while excluding secrets and raw sensitive content. Configure a collector/exporter and dashboards in the deployment; instrumentation without collection provides incomplete visibility.

Why: audits support accountability; telemetry supports debugging and reliability. Mixing them risks either missing decisions or logging confidential data excessively.

Verify: trace one incident from its API request to worker completion, inspect the audit actor and run integrity checks. Confirm credentials are absent from logs.

Step 15 — Evaluate before claiming quality

The supplied synthetic suite records expected causes, evidence, tools, recommendations and approval requirements. The evaluator runs the actual workflow and calculates observed metrics. The console shows measured results only when a report exists.

Separate deterministic fixture accuracy from live-model accuracy. Synthetic latency is not production P95 latency. Simulated remediation is not real rollback reliability. Token cost is unknown unless token usage and model pricing are recorded; do not assign a fictional dollar value.

Why: evaluation makes regressions visible and gives an interview discussion evidence. Honest scope matters more than impressive-looking scores.

Verify: run the eval command, open `evals/results/latest.json`, inspect failed cases and compare its report to the UI. See [evaluation.md](#).

Step 16 — Package reproducibly

Compose starts the database, API, worker and web app with health checks. The API container and worker container use the same code image and different commands. Named database volumes retain data when containers are recreated. `.env.example` describes expected settings; never commit `.env` secrets.

CI should install known dependencies, run backend safety tests and evals, then type-check/build the frontend. A passing build does not prove production readiness; it proves the configured checks passed for that revision. Treat dependency upgrades as reviewable changes and regenerate locks intentionally.

Why: a project that runs only on its author's machine is difficult to review and operate. Repeatable packaging makes demonstration and deployment practical.

Verify: use the README quick start and CI workflow. Docker execution may depend on local daemon access; the validation report states what was actually executed in this environment.

Step 17 — Prepare a real deployment

Choose one cloud first. The documented GCP path maps the web/API to Cloud Run, PostgreSQL to Cloud SQL, credentials to Secret Manager, and the worker to an always-on service or dedicated worker runtime. Optional Redis is managed separately. GitHub Actions should authenticate using workload

identity instead of storing a long-lived cloud key.

Before production, implement enterprise OAuth/OIDC with issuer/audience validation and key rotation, service-scoped RBAC, trusted webhook ingestion, migration rollout, backup restore tests, worker lease monitoring, adapter-specific idempotency, egress restrictions, secret rotation, load tests and incident handling. Tenant isolation is necessary if serving multiple customers. Terraform can encode the reviewed cloud resources after the architecture and ownership rules are settled.

Why: production readiness is an operational commitment, not a framework list. Provisioning resources does not supply safe permissions, restore procedures or a tested integration.

Verify: follow [deployment.md](#), perform a staging exercise and a database restore drill, and complete the production requirements in the README. No cloud account was provisioned or deployed by this build.

Project structure and how to navigate it

SentinelOps/	
├─ README.md	Start, scope, demos and documentation map
├─ compose.yaml	Local web/API/worker/PostgreSQL topology
├─ .env.example	Documented runtime configuration
├─ Makefile	Shortcuts for repeatable developer tasks
├─ apps/	
│ └─ api/	
│ │ └─ pyproject.toml	Python dependencies and tool configuration
│ │ └─ Dockerfile	API/worker image
│ │ └─ alembic/	Versioned database schema migrations
│ │ └─ sentinelops/	
│ │ └─ main.py	FastAPI app and HTTP routes
│ │ └─ config.py	Typed environment settings
│ │ └─ auth.py	Token validation and role dependencies
│ │ └─ models.py	SQLAlchemy persistence models
│ │ └─ db.py	Database engine/session setup
│ │ └─ workflow.py	LangGraph investigation and policy gate
│ │ └─ adapters.py	Typed fixture observation/mutation adapters
│ │ └─ policy.py	Allowlist, argument validation, approval rules
│ │ └─ worker.py	Durable job claim/process/recovery loop
│ │ └─ audit.py	Append-only events and integrity verification
│ │ └─ runbooks.py	Curated runbooks and retrieval
│ │ └─ seed.py	Idempotent runbook seed command
│ │ └─ migrate.py	Migration entrypoint
│ │ └─ mcp_server.py	Read-only MCP tool interface
│ │ └─ evaluate.py	Measured synthetic evaluation runner
│ └─ tests/	Backend unit and safety invariants
│ └─ web/	
│ │ └─ app/	Next.js app entrypoint and styling
│ │ └─ public/	Favicon and static assets
│ │ └─ package.json	Frontend scripts/dependencies
│ │ └─ package-lock.json	Exact npm dependency resolution
│ └─ Dockerfile	Next.js standalone container
├─ tests/	Workflow/API integration and security checks
├─ evals/	
│ └─ incidents.json	Synthetic observations and expected outcomes
│ └─ build_fixtures.py	Reproducible corpus construction
│ └─ results/latest.json	Generated measured report; not a made-up score
├─ infra/	Cloud/telemetry configuration and references
├─ .github/workflows/	Continuous integration
└─ docs/	
│ └─ architecture.md	Responsibilities, diagrams and trust boundaries
│ └─ learning-guide.md	Every implementation step and its purpose
│ └─ project-structure.md	This navigation guide
│ └─ security.md	Permissions, approvals and threat boundaries
│ └─ evaluation.md	Methodology and metric limitations
│ └─ deployment.md	Local/cloud deployment instructions
│ └─ operations.md	Monitoring, recovery and maintenance
│ └─ validation.md	What actually passed in this workspace
└─ adr/	Architecture decision records

The exact tree may contain additional small helper modules. Use `rg --files` to list them. Generated dependencies (`node_modules` , `.venv`), local SQLite databases, secrets and temporary build files are intentionally excluded from the source deliverable.

Read the code in this order

1. `config.py` : understand demo, model and database configuration. Changing a provider key should not require a source edit.
2. `models.py` : understand what survives a process restart. Inspect incidents, proposals, approvals, jobs and audit records.
3. `policy.py` : understand what the system is permitted to do. This is the authority boundary, not a model instruction.
4. `adapters.py` and `runbooks.py` : understand what observations exist and how simulation differs from a real connector.
5. `workflow.py` : follow the steps from observations to a recommendation.
6. `worker.py` : see how durable requests become work, and how failures are handled.
7. `main.py` and `auth.py` : follow request validation, authentication and role checks.
8. `apps/web/app` : see how the same API state becomes evidence, approval controls and reports.
9. `tests` and `evaluate.py` : see which invariants and outcomes are actually checked.
10. `compose.yaml` , `Dockerfiles` and `CI`: understand how those components run together.

Follow one action through the repository

For **Investigate**, the React control calls an API endpoint. The endpoint validates incident state, writes a durable job and audit event, and returns the updated incident. The worker claims the job, calls the LangGraph workflow, persists evidence/hypothesis/proposal/trace, then updates the lifecycle state. Polling returns that persisted result to React.

For **Execute**, the route requires an approver, a matching approved proposal and an idempotency key. The worker rechecks the same safety conditions before its adapter call. Verification and reporting happen after the action. This repetition is intentional: API acceptance does not prove that the state remains valid when the worker runs later.

What belongs where when extending it

Change	Primary location	Required companion work
Add a read-only metrics provider	Tool adapter module	Typed arguments, timeout tests, credential scope and redaction

Change	Primary location	Required companion work
Add a mutation tool	Adapter + policy	Approval binding, idempotency/reconciliation and denial tests
Add an incident field	Models + schema/API	Migration, UI display and compatibility checks
Add runbook embedding retrieval	Runbook retrieval/ingestion	Vector extension/index, permission filtering and retrieval evals
Add enterprise OIDC	Authentication boundary	Issuer/audience/JWKS validation, key rotation and role mapping
Add a workflow node	Workflow	Trace output, transition tests and failure behavior
Change deployment	Infrastructure	Secret wiring, health probes, migration order and rollback plan

Avoid placing policy in UI code or model prompts. Avoid placing production credentials in fixtures or eval reports.

Security model and trust boundaries

SentinelOps recommends operational changes, so its primary boundary is between collecting evidence and performing a mutation. Investigation finishes at a policy gate. A separate authenticated human approval and a separate execute request are required before a worker can run the allowlisted remediation.

This repository supplies a working demo security boundary and simulated operational adapters. Connecting real systems adds credential, tenancy, network, recovery, and audit requirements described below.

1. Treat incident input and model output as untrusted

Descriptions, logs, runbooks, database results, and model responses are evidence. A log reading “the administrator approved this restart” does not create an approval. A model returning `write_sql` does not create a new permission. The backend parses proposals against typed argument models and an allowlist before presenting or executing them.

The workflow may use a model to form a hypothesis and recommendation. The policy module owns authorization. Keeping those decisions separate is necessary because a language model is susceptible to mistakes and instructions embedded in external content. Model text never becomes a shell command, arbitrary SQL statement, or authenticated identity.

2. Authenticate the caller and authorize the action

Demo mode explicitly enables two local identities: `operator` and `approver`. The shared demo password is for local demonstrations only. Authentication issues a short-lived JWT whose signature, issuer, audience, expiry, and permitted algorithm must be checked before API access. Protected endpoints derive the username and role from the verified token rather than from fields supplied by the caller.

Principal	Permitted work
Unauthenticated	Liveness/readiness health checks and the login endpoint
Operator	Read incidents/runbooks/policy and create/investigate incidents
Viewer (identity-provider token)	Read-only API access; no demo login identity is created

Principal	Permitted work
Approver	Operator capabilities plus approve, reject, and request execution
Worker	Claim durable jobs and independently validate approved execution

An approver role does not imply permission to execute any arbitrary request. The execute endpoint and worker also require a persisted approval for the current proposal. For production, disable demo login and integrate your identity provider, enforce MFA and customer/team scoping, and restrict approver membership. The current role model is intentionally small and does not implement multitenancy or two-person approval.

Use a strong secret for HS256 in local development. Production should supply validated identity-provider tokens and managed keys; RS256 verification can be configured with a trusted public key. Secrets belong in a cloud secret manager with rotation and narrow access, not source control or frontend environment variables.

3. Classify and constrain each tool

Tools	Risk	Enforcement
Recent deployments, metrics, logs, runbook retrieval	Low	Typed read arguments; no approval
Named database diagnostics	Medium	Adapter-owned templates; no raw SQL
Restart service, roll back deployment	High	Exact proposal approval, role checks, durable execution claim
Write SQL, delete data, unknown tools	Blocked	Rejected before adapter dispatch

Sensitive database reads can warrant a higher classification in a real customer deployment. The demo's aggregate diagnostics are medium risk and do not require mutation approval. Policy should reflect actual data exposure and credentials.

`validate_tool()` rejects unknown arguments, invalid types, invalid service identifiers, overlong values, and out-of-range limits. Database requests accept only `connection_stats` or `slow_queries`, never model-supplied SQL. Live implementations must use bound parameters and a separate read-only database principal with access restricted to necessary views. Named templates alone do not protect a broadly privileged database credential.

4. Bind approval to the exact action

The backend computes a SHA-256 fingerprint over the normalized tool name and arguments. Approval persists the proposal ID, fingerprint, authenticated approver, reason, and timestamp. Execution recomputes the fingerprint and verifies:

1. The caller has the approver role.
2. The proposal is currently approved.
3. Its tool and arguments are still allowlisted and strictly typed.
4. The persisted approval refers to that proposal and that fingerprint.
5. The approval identifies a human principal and has valid timestamps.
6. The idempotency key is valid and agrees with any already-bound key.

Changing the service or deployment target invalidates the original approval. An authenticated approver must review a new proposal. The fingerprint binds content; it is not a cryptographic signature from an external identity provider. A database administrator able to rewrite both proposal and approval remains inside the database trust boundary.

5. Separate approval, scheduling, execution, and verification

Approval does not run a tool. The execute request creates a durable execution record and queues a job. The worker rechecks authorization immediately before adapter dispatch. Database uniqueness constraints prevent a second execution for the same proposal and bind an idempotency key to the incident. Repeating the same request should return its existing execution rather than perform another mutation; using a different key cannot authorize a replay.

Database job claims and leases make work survive an API restart. PostgreSQL locking is necessary when multiple workers compete. SQLite supports the local single-worker demonstration but should not be used to infer multi-worker production concurrency guarantees.

A database transaction cannot atomically commit a change in an unrelated cloud service. A worker might crash after the provider accepted a mutation but before recording its receipt. Live adapters therefore need provider-side idempotency, durable receipts, and reconciliation of uncertain outcomes. Treat a high-risk execution with an uncertain result as requiring attention instead of automatically replaying it.

After an adapter reports success, verification checks the resulting service health. A failed verification must keep the incident unresolved and visible for human action. Demo recovery metrics are simulated; a live adapter needs independent metrics and a defined observation window.

6. Make audit history tamper evident

Audit events record the actor, action, time, proposal identifiers, and relevant details. Each event links to the previous event's hash, creating a chain. This makes accidental or local edits detectable when the chain is validated and its earlier head is trusted.

The application appends events; it does not expose an audit-edit endpoint. That is an application-level append-only convention. A database administrator can delete events or rewrite the complete chain. Hashes stored beside the records do not make the database immutable.

For stronger production evidence, write audit records to a separate restricted sink, retain periodic signed chain heads outside the application database, use object retention/WORM controls where required, and alert on gaps or integrity failures. Avoid storing raw credentials or unnecessary personal data in audit details.

7. Keep live credentials behind the backend boundary

Browser code receives only the API URL and its user session token. Operational credentials stay on the backend/worker side. Limit CORS origins to the deployed UI and use TLS. CORS is a browser policy, not authentication; non-browser callers can still reach public endpoints and must pass JWT checks.

The OpenAI integration sends evidence to the configured provider only in explicit live-model mode. Review data handling and redaction for real incidents. A production MCP adapter must pin trusted servers, authenticate its transport, validate tool responses, scope credentials by customer and service, and preserve the same local execution gate. An MCP connection does not automatically make a tool safe.

Demo mutations change simulated state. Live adapters fail closed until implemented and configured. A flag must not turn a simulator into a privileged live operator. Before enabling a real integration, test denied permissions, credential expiry, network failures, rate limits, partial execution, and reconciliation in staging.

8. Deploy with least privilege and operational controls

Keep the database and internal services private. Separate API, worker, and migration identities; restrict who can approve, who can execute, and who can change policy. Use managed backups, restore drills, secret rotation, vulnerability updates, request rate limits, bounded telemetry payloads, and an incident response process for this platform itself.

The supplied Docker Compose environment is a reproducible local deployment. It does not provide public HTTPS, a production identity provider, managed database backups, customer isolation, external immutable audit retention, or cloud-specific runtime hardening. Those are concrete deployment work, not properties obtained from choosing FastAPI, LangGraph, or containers.

See [evaluation.md](#) for the measured safety probes and lifecycle tests. Their passing results validate the tested paths; production assurance also depends on the real adapters, infrastructure configuration, operator procedures, and data boundaries.

Evaluating SentinelOps

An incident agent can sound convincing while naming the wrong cause or recommending a dangerous tool. Evaluation turns those behaviors into reviewable measurements. The supplied numbers are generated by running code; the repository does not assume target scores.

1. Freeze the questions before measuring the answers

`evals/incidents.json` contains 64 synthetic incidents: 16 release-related failures, 16 memory-growth incidents, 16 saturated database pools, and 16 ambiguous or healthy observations. They cover eight service names and different error rates, release ages, resource pressures, connection limits, misleading historical releases, and hostile log text.

Each fixture contains two sections:

Section	Contents	Who receives it
input	Service, description, explicit metrics/deployments/logs/database observations	Investigation workflow
expected	Root cause, relevant evidence sources and runbook IDs, allowed read tools, forbidden tools, recommendation, approval requirement	Evaluator only

Every input has `scenario: "unknown"`. This matters: passing a cause-specific scenario such as `memory_leak` to the classifier would give it the answer. The fixture tools expose the supplied observations; the classifier must derive its answer from those observations. `workflow_input()` enforces the accepted input fields and rejects a non-neutral scenario.

`evals/build_fixtures.py` reconstructs the checked-in JSON from explicit observation rows. Review the JSON, freeze it for a comparison, and change its version deliberately when you change the benchmark. The report records its SHA-256 hash so two runs can be compared against the same data.

These are integration fixtures designed alongside the rule baseline. They are useful regression checks, but they are not an independent test set proving generalization to real incidents.

2. Run the real workflow

From the repository root, install the backend and run:

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -e 'apps/api[dev]'
cd apps/api
python -m sentinelops.evaluate --output ../../evals/results/latest.json
```

The harness calls `sentinelops.workflow.investigate_case()` for every case, with fixture adapters and real orchestration/retrieval/policy code. It records wall-clock duration using a monotonic clock. A case that raises an exception stays in the denominator and scores zero on the correctness checks; it is not silently excluded.

The default `--mode fixture` avoids network calls and API credentials. Its output identifies itself as `deterministic_fixture_baseline`. A high score means that the rules and their integration handle these observations; it does not mean an OpenAI model has that accuracy.

For a separate, explicitly billed live-model run, configure `OPENAI_API_KEY` and `OPENAI_MODEL`, then run:

```
python -m sentinelops.evaluate --mode openai --output ../../evals/results/openai.json
```

This still uses synthetic telemetry. Keep the output separate from `latest.json` and compare it with the deterministic baseline. A live-model request or parsing failure counts as an error; the evaluator must not relabel a fixture fallback as a measured live-model result. The interface and code should be inspected before sending real incident data to an external model.

3. Understand exactly what each metric measures

Metric	Formula / observation	Practical meaning
Root-cause accuracy	Correct expected root labels / all 64 cases	Correctness within the four supplied categories
Retrieval Recall@5	Mean of relevant expected runbooks found among the first five results / expected relevant runbooks	Whether the relevant seeded runbook was available
Evidence coverage	Mean of expected evidence source types present / expected source types	Whether required types of evidence were gathered
Tool-selection accuracy	Expected mutation tool equals proposed mutation tool, including null / all cases	Whether the action recommendation is appropriate for the fixture

Metric	Formula / observation	Practical meaning
Read-tool policy adherence	All recorded investigation calls use validated accepted read tools / all cases	Whether investigation remained inside the read boundary
Unsafe-action rejection	Rejected hostile tool inputs / hostile tool probes	Allowlist and argument validation behavior
Approval-policy adherence	Correct fixture approval gates plus denied unauthorized execution probes / both groups combined	Proposal gates and independent authorization rechecks
Accepted-control accuracy	Valid calls accepted / valid controls	Detects a broken policy that rejects everything
Latency P50/P95	Nearest-rank percentiles of local per-case wall time	Repeatable local investigation overhead, subject to host load

Approval-policy adherence combines two different kinds of checks. The report preserves per-case outcomes and individual policy probes so a reviewer can inspect their denominators separately. Safety probes include unknown tools, write/delete tools, raw SQL, extra fields, invalid template names, wrong roles, missing/expired approvals, changed tool arguments, changed fingerprints, and changed bound idempotency keys.

The report deliberately sets `successful_remediation` and `average_llm_cost_usd` to `null`: this harness stops at the approval gate and does not collect provider billing or execute approved remediations. Demo remediation and verification are exercised in the lifecycle tests. Their success is simulator behavior, not a production recovery rate.

Several tempting interpretations are incorrect:

- Source presence does not prove that the explanation accurately cites its evidence. Add human or structured evidence-grounding review for that.
- Retrieving among four runbooks can yield generous Recall@5. Evaluate a larger, distractor-rich collection before selecting a production retrieval strategy.
- Rule-generated confidence is not calibrated probability. Do not display it as a measured likelihood of correctness.
- Local fixture latency does not include real telemetry providers, network queues, model calls, or customer database performance.
- One adversarial log example does not establish resistance to all prompt injections. Maintain an expanding attack suite and keep policy enforcement outside the model.

4. Verify the full human approval lifecycle

From the repository root:

```
python -m pytest tests apps/api/tests
```

The tests call FastAPI through its test client, use a temporary SQLite database, and drain queued jobs explicitly. No production system is contacted. Their purpose is to check boundaries that a workflow-only score cannot see: unauthenticated requests, wrong roles, execution before approval, rejected proposals, fingerprints changed after approval, duplicate idempotency keys, and simulated verification failure.

The policy unit tests also check strict typed arguments and named read-only query templates. A read-only string beginning with `SELECT` is not sufficient protection: it could still expose data or contain unsafe SQL. SentinelOps instead accepts adapter-owned template names and rejects SQL strings entirely.

5. Add stronger evaluation before customer use

Begin with anonymized historical incidents reviewed by operators. Include incidents the agent should refuse to diagnose, missing telemetry, competing hypotheses, contradictory evidence, stale runbooks, compromised logs, and provider outages. Keep development and holdout sets separate. Record the model, prompt, tool schemas, retrieval version, and dataset hash with each run.

Next, test model calls with real usage records so token count and cost are measurable. Add calibrated confidence and evidence-grounding review, and measure a complete approved remediation in a controlled staging system. Successful recovery should mean the system stayed healthy for a defined observation window, not simply that a mutation API returned 200.

Finally, set release gates appropriate to the customer. Safety failures should block a release. Accuracy thresholds need a baseline, a representative held-out dataset, and an explicit tolerance for abstaining. The included CI gates validate this implementation and its fixtures; they do not certify production readiness.

Run and deploy SentinelOps

The supplied implementation runs a complete local demonstration: browser, API, durable investigation/execution worker, PostgreSQL and approval workflow. It uses simulated operational evidence and mutations by default. A production deployment needs an identity provider, real read-only adapters, reviewed remediation adapters and operations controls. No cloud resources have been created by this project.

1. Understand what runs where

Component	Local runtime	Purpose and reason
Browser UI	Next.js, port 3000	Lists incidents, shows evidence and lets authorized humans review proposals.
API	FastAPI, port 8000	Validates input, authenticates callers, enforces roles and saves state.
Worker	Separate Python process	Runs slow investigations and approved execution without occupying an HTTP request.
PostgreSQL	PostgreSQL 16 with pgvector	Persists incidents, jobs, proposals, evidence and audit events across process restarts.
Migration task	One-off Python process	Installs versioned schema changes before API/worker start. Seeds runbook content.
Redis	Optional cache profile	Future cache/rate-limit infrastructure; the current queue does not use Redis.
OpenTelemetry collector	Optional telemetry profile	Receiver scaffold for future instrumentation; current workflow traces are database records.

```

flowchart LR
    Browser[Browser] -->|HTTP localhost:3000| Web[Next.js web]
    Browser -->|Bearer JWT HTTP localhost:8000| API[FastAPI API]
    API -->|Persist jobs and approvals| DB[(PostgreSQL + pgvector)]
    Worker[Durable worker] -->|Claim jobs using locks and leases| DB
    Worker -->|Read fixture evidence / simulate mutation| Tools[Demo tools + LangGraph workflow]
    Init[Migration + runbook seed] --> DB
    Redis[(Optional Redis cache)]
    OTEL[Optional telemetry receiver]

```

The browser connects directly to the API. `NEXT_PUBLIC_API_URL` must therefore be an address the browser can reach; `http://api:8000` is only a Docker network address and will not work from a host browser. Next.js embeds `NEXT_PUBLIC_*` values during the build, so changing an API URL requires rebuilding the web image.

2. Install the local prerequisites

Use Git and Docker Engine/Desktop with Compose v2. The native alternative needs Python 3.12 and Node.js 22. Docker provides consistent PostgreSQL, Python and Node runtimes without installing each database/runtime on your machine. Your machine needs access to npm, PyPI and container registries for the first build.

3. Choose explicit local configuration

From the repository root:

```

cp .env.example .env
docker compose config --quiet

```

The sample contains clearly labeled development credentials. `DEMO_MODE=true` enables `operator` and `approver` accounts with password `sentinel-demo`; `LLM_MODE=fixture` uses deterministic fixture reasoning without a paid model key. All live mutations remain disabled. Configuration validation catches malformed Compose before building; `--quiet` avoids displaying rendered credential values.

The checked-in signing key is long enough for the demo but is public knowledge. It does not provide security for an Internet-facing service. Keep the default loopback bindings while using demo credentials. Changing the password in `.env` does not change credentials inside an already initialized PostgreSQL volume; use an explicit database password rotation or reset the local volume.

4. Build and start the full stack

```
docker compose up --build -d --wait
```

Compose waits for PostgreSQL, runs the schema migrations and runbook seed, then starts API, worker and web. Health checks verify API database readiness, worker process liveness and the frontend HTTP response. `make up` is the same command.

Open `http://localhost:3000`. API documentation is at `http://localhost:8000/docs`. Only web/API ports are published by default, both on loopback. PostgreSQL and Redis stay on the private Compose network. The PostgreSQL named volume survives ordinary stops; the optional Redis volume is a cache volume. Source code is baked into images, so code changes require a rebuild.

5. Verify the complete workflow

```
python infra/smoke_test.py
```

This script uses the HTTP API to create a checkout incident, waits for the worker, checks the expected hypothesis, confirms an operator cannot approve, approves as the approver, queues execution and checks simulated resolution. It repeats the same execution key to exercise idempotency. It leaves one labeled test incident in the demo database so you can inspect its evidence and audit trail in the UI.

This is different from a page-load check: it verifies that the queue, worker, policy and database cooperate through their real service boundary.

6. Run tests and measured evaluations

For native developer tooling, create the virtual environment in `work/`:

```
python3.12 -m venv work/venv
. work/venv/bin/activate
python -m pip install --require-hashes -r apps/api/requirements-dev.lock
python -m pip install --no-deps --no-build-isolation -e apps/api
python -m pytest tests/apps/api/tests -q
cp .env.example apps/api/.env
cd apps/api
python -m sentinelops.evaluate --output ../../evals/results/latest.json
```

The runtime and developer lockfiles pin the same runtime versions and include artifact hashes. The developer lock adds test tooling. CI installs those exact dependencies, runs both test suites and measured fixture evaluations, checks TypeScript, builds Next.js, then builds/runs the complete

Compose stack and smoke test. The fixture evaluation report measures this simulated baseline; it is not a claim about model accuracy against real incidents.

Compose mounts `evals/results/` read-only into the API, so a newly generated report is available through the evaluations endpoint without rebuilding the API image. The mounted directory must exist and have host-readable permissions. A cloud image includes the report present at image build time instead of this local mount.

7. Run without Docker when learning each process

After installing the locked Python dependencies, copy `.env.example` into `apps/api/.env`. That location matters: settings read `.env` relative to the process working directory. The copied sample selects SQLite, disables pgvector and points evaluations to the root report.

In separate terminals, activate the same virtual environment and run:

```
# Terminal 1, from apps/api
python -m sentinelops.migrate
python -m sentinelops.seed
uvicorn sentinelops.main:app --host 127.0.0.1 --port 8000

# Terminal 2, also from apps/api
python -m sentinelops.worker

# Terminal 3, from apps/web
npm ci
npm run dev
```

SQLite is a convenient single-machine fallback. It is not the deployment choice for concurrent API/worker replicas; PostgreSQL row locking is necessary for that mode. The same worker/application separation is retained even in native mode.

8. Stop, inspect, or intentionally reset

```
docker compose logs --tail=100 api worker migrate
docker compose down
```

Ordinary `down` keeps database data. `docker compose down --volumes` intentionally deletes local database/cache data; use it only when you want a fresh demo. See `operations.md` for failure recovery and backup guidance.

Production architecture on Google Cloud

flowchart LR

```
User[Browser] -->|HTTPS| Web[Cloud Run web]
User -->|HTTPS + organization-issued JWT| API[Cloud Run API]
API -->|Cloud SQL connector socket| SQL[(Cloud SQL PostgreSQL)]
Worker[Cloud Run worker: CPU always allocated, minimum 1] --> SQL
Worker --> Adapters[Reviewed adapters to real operational services]
API --> Secrets[Secret Manager]
Worker --> Secrets
Migration[Cloud Run migration job] --> SQL
Registry[Artifact Registry: immutable images] --> API
Registry --> Web
Registry --> Worker
Redis[(Optional Memorystore Redis)]
Logs[Cloud Logging + monitoring]
API --> Logs
Worker --> Logs
```

Cloud Run supplies HTTPS and managed container scheduling. Cloud SQL supplies a managed PostgreSQL database with backup/PITR options. Artifact Registry holds versioned container images. Secret Manager supplies credentials without putting them in git or browser bundles. Separate service accounts limit each component's cloud permissions. These are infrastructure capabilities, not substitutes for application approval checks or evidence quality.

The database-polling worker cannot use request-only CPU or scale to zero: no incoming HTTP request wakes it when an API inserts a database job. Its template sets CPU always allocated and a minimum of one instance. The included `infra/worker_service.py` wrapper starts the worker and binds Cloud Run's `PORT` for health checks. This adds an always-running worker cost. A later queue-triggered design could use Cloud Tasks/Pub/Sub with an explicit delivery/lease contract; those integrations are not implemented here.

Production prerequisites, in order

1. **Identity:** disable demo login, configure an organization token issuer and RS256 public verification key, and map signed `sub / role` claims to the allowed roles (`viewer`, `operator`, `approver`). Tokens must also contain `iat`, `nbft`, `exp`, matching `iss` and `aud=sentinelops-api`. The sample UI currently supports demo login, so a real SSO/token acquisition flow must be added before deployment is useful to end users. Never give the API an identity provider's private key. JWKS fetching/automatic key rotation is not implemented.
2. **Live evidence adapters:** replace fixture evidence with scoped monitoring, logs, deployment and read-only database adapters; test access in staging.
3. **Remediation adapters:** implement allowlisted typed live operations, upstream idempotency/reconciliation and least-privilege credentials. The existing live mode fails closed.

Setting an environment flag does not create these adapters.

4. **Database and IAM:** choose project/region, create Cloud SQL PostgreSQL 16, database roles, backup retention and service accounts. Use an owner/migration database account for DDL and a restricted runtime account for normal DML.
5. **Operational controls:** review load/cost limits, alerting, data retention, encrypted external audit retention and restore drills. A database hash chain alone does not stop an administrator rewriting history.

The supplied manifests deliberately set `DEMO_MODE=false` and `LIVE_TOOLS_ENABLED=false`. They are a deployment scaffold for completing these prerequisites, not a claim that the prototype is production-ready.

Reviewable Cloud Run deployment steps

These commands describe actions to run in your own authorized GCP deployment session. They have not been executed against a cloud project here.

1. Create an Artifact Registry Docker repository and Cloud SQL instance in your chosen region. Enable Cloud Run, Artifact Registry, Cloud SQL Admin and Secret Manager APIs. Create API, worker, web and migration service accounts. Grant API/worker/migration `roles/cloudsql.client`; grant secret access only to each account's relevant secret. The web account needs no SQL or credential access. Cloud SQL IAM grants connector access; PostgreSQL role privileges independently control what SQL that connection can execute. If your instance uses only a private IP, add the required VPC/Direct VPC egress configuration; the supplied socket annotation alone does not configure that network path.
2. Create database users/roles. Use a separate migration database owner and a runtime role with only the required table privileges. Grant runtime privileges after initial migrations and maintain those grants for future tables. Install the supported `vector` extension as the database owner if you intend to develop embedding retrieval; baseline retrieval works without it.
3. Store runtime and migration database connection strings in separate Secret Manager secrets. A Cloud SQL Unix socket DSN has this form; URL-encode username and password characters before constructing it:

```
postgresql+psycpg://USER:URL_ENCODED_PASSWORD@/sentinelops?  
host=/cloudsql/PROJECT:REGION:INSTANCE
```

Store the IdP RS256 public key in the verification-key secret. Use explicit secret version numbers for repeatable releases. Do not paste secrets into rendered YAML or command histories.

4. Build and push the API image and web image. The web build must know the final public API origin; reserve/map a stable origin or obtain the API service URL before building web. Prefer immutable image digests in manifest variables:

```
export API_BUILD_TAG='REGION-docker.pkg.dev/PROJECT/REPOSITORY/api:RELEASE'
export WEB_BUILD_TAG='REGION-docker.pkg.dev/PROJECT/REPOSITORY/web:RELEASE'
export API_ORIGIN='https://YOUR-API-ORIGIN'
gcloud auth configure-docker "$REGION-docker.pkg.dev"
docker build -f apps/api/Dockerfile -t "$API_BUILD_TAG" .
docker push "$API_BUILD_TAG"
docker build --build-arg NEXT_PUBLIC_API_URL="$API_ORIGIN" -t "$WEB_BUILD_TAG" apps/web
docker push "$WEB_BUILD_TAG"
```

5. Resolve the pushed image digests in Artifact Registry, then set non-secret variables for immutable image references, resource IDs and version numbers:

```
export API_IMAGE='REGION-docker.pkg.dev/PROJECT/REPOSITORY/api@sha256:DIGEST'
export WEB_IMAGE='REGION-docker.pkg.dev/PROJECT/REPOSITORY/web@sha256:DIGEST'
export CLOUD_SQL_CONNECTION_NAME='PROJECT:REGION:INSTANCE'
export API_SERVICE_ACCOUNT='sentinelops-api@PROJECT.iam.gserviceaccount.com'
export WORKER_SERVICE_ACCOUNT='sentinelops-worker@PROJECT.iam.gserviceaccount.com'
export WEB_SERVICE_ACCOUNT='sentinelops-web@PROJECT.iam.gserviceaccount.com'
export MIGRATION_SERVICE_ACCOUNT='sentinelops-migrate@PROJECT.iam.gserviceaccount.com'
export DATABASE_SECRET_VERSION='1'
export MIGRATION_DATABASE_SECRET_VERSION='1'
export JWT_KEY_VERSION='1'
export JWT_ISSUER='https://YOUR-IDENTITY-ISSUER'
export WEB_ORIGIN='https://YOUR-WEB-ORIGIN'
python infra/gcp/render.py --output work/gcp-rendered
```

6. Review the rendered manifests. Create/execute the migration job before serving traffic, then deploy API, worker and web in that order:

```
gcloud run jobs replace work/gcp-rendered/migrate.job.yaml --region "$REGION" --project
"$PROJECT_ID"
gcloud run jobs execute sentinelops-migrate --wait --region "$REGION" --project "$PROJECT_ID"
gcloud run services replace work/gcp-rendered/api.service.yaml --region "$REGION" --project
"$PROJECT_ID"
gcloud run services replace work/gcp-rendered/worker.service.yaml --region "$REGION" --project
"$PROJECT_ID"
gcloud run services replace work/gcp-rendered/web.service.yaml --region "$REGION" --project
"$PROJECT_ID"
```

7. Set invocation IAM deliberately. The direct-browser design requires publicly reachable web/API at the Cloud Run layer, while the API still enforces JWTs. An organization requiring Cloud Run IAM/IAP must add a trusted session/token proxy or compatible gateway flow; browser app JWTs do not automatically satisfy Cloud Run IAM. Keep the worker internal with no public invoker permission.

8. Check readiness, submit a staging incident using real signed tokens, verify role denial/approval, and confirm the worker processes jobs while receiving no requests. Confirm backup restore, alert delivery and identity key rotation before sending production traffic. Do not run the demo smoke script against a production service; demo login must be disabled there.

Memorystore Redis is optional and unused by this implementation. If future code uses it, add private networking/Direct VPC egress, authentication where supported, memory limits and cache eviction policy. Do not introduce an extra managed service solely because a reference diagram contains Redis.

Terraform is a suitable later layer for provisioning project services, IAM, SQL, Secret Manager references, registries and networking. Region, availability, budget and retention must be chosen first; the included Cloud Run YAML templates do not claim to provision that entire platform or to have passed a cloud apply.

Operate and troubleshoot SentinelOps

An API returning 200 is not enough. The API must reach the database, workers must claim jobs, proposals must remain unchanged between approval and execution, and the audit trail must record each transition. The local stack demonstrates those boundaries; production integrations need additional provider-side reconciliation.

Health and progress

Signal	Meaning	Limitation
API /health/live	HTTP process responds	Does not prove database or queue availability.
API /health/ready	API can query the database	Does not prove a worker is consuming jobs.
Worker wrapper /health/live	Supervised worker process is alive	Does not prove a job is making progress.
Queued-job age / expired leases	Work is waiting or stalled	Requires a monitoring query/exporter in production.
Incident trace + audit	Stored workflow and authorization history	Needs external retention for stronger tamper resistance.

The prototype does not ship a worker-heartbeat exporter, Prometheus endpoint or complete remote OpenTelemetry pipeline. Application logs are structured JSON; `OTEL_CONSOLE_EXPORT=true` optionally exports workflow spans to console output. The optional collector is a receiver scaffold, and no traces are automatically exported to it. Database workflow traces are the UI's current source of step timing and status.

Useful local commands

```
docker compose ps
docker compose logs --tail=100 api worker migrate
curl --fail http://127.0.0.1:8000/health/ready
python infra/smoke_test.py
```

For database inspection without publishing a database port:

```
docker compose exec postgres psql -U sentinelops -d sentinelops
```

If a host tool needs a port, opt into the loopback override:

```
docker compose -f compose.yaml -f infra/compose.debug.yaml --profile cache up -d
```

The optional Redis cache is not required to process incidents. Restarting it has no effect on the database-backed job queue in this version.

When something fails

Symptom	What to check	Why
Browser cannot reach API	Browser-visible NEXT_PUBLIC_API_URL; rebuild web; exact CORS_ORIGINS	Next public variables are embedded at build time, and CORS matches full origins.
Login returns 403	DEMO_MODE and intended deployment identity	Demo login is intentionally unavailable outside explicit demo mode.
Login/auth returns 503	Signing algorithm and verification-key configuration	Authentication fails closed rather than accepting unverifiable tokens.
Migration container exits	Migration logs, DB credentials, extension permissions	API/worker wait for the schema task to succeed.
Incident stays investigating/executing	Worker process, job status/lease, adapter timeout	API only queues work; a stalled worker cannot be fixed by reloading the page.
API ready becomes 503	DB availability, network/socket, credential rotation	Readiness checks include a database query.
Live action fails	Missing reviewed live adapter	Flags cannot turn a simulated adapter into a production operation.
Evaluations show not run	EVAL_REPORT_PATH, report existence and readability	The API reads a measured report artifact, not invented percentages.

Do not repeatedly queue execution or alter database status fields to bypass a failed approval. Read the incident's trace, proposal, audit and job state first. Keep the original idempotency key when retrying the same execution request.

Queue recovery and mutation boundaries

Investigation and execution jobs are stored in PostgreSQL. Worker claims use locks and leases; explicit retry/recovery logic distinguishes read-only work from potentially ambiguous mutation outcomes. A process crash during investigation can be retried within those boundaries. A crash after a live provider changed state but before the application recorded success cannot be made safe by a database lease alone. Future adapters must support an upstream idempotency key or reconcile the actual provider state before retrying. An ambiguous live outcome needs human review.

The same distinction applies during rolling deployments. Even with a maximum of one worker replica, old and new revisions can overlap during replacement. Locks prevent concurrent claims of one healthy leased job; they do not constitute a global exactly-once promise for arbitrary remote APIs.

The worker stops taking new jobs on SIGTERM and attempts to finish the current job. Container/cloud shutdown deadlines can still interrupt long-running work. Bound adapter calls with timeouts, make lease duration/renewal consistent with workload duration, and test interruption at each mutation boundary before integrating a real remediation tool.

Approvals and audit

Approve only after reviewing evidence and typed tool arguments. Approval records the exact proposal fingerprint. Execution independently checks the caller's role, approval, fingerprint, policy and idempotency key. Approval itself does not execute. Rejecting a proposal leaves a reviewable record rather than silently changing it.

Audit events are hash chained to expose changes/reordering during verification. PostgreSQL triggers reject update, delete and truncate operations on audit events through the normal database path. A privileged database administrator can disable those controls, rewrite the events and recompute the chain. For production, export to independently controlled append-only storage with retention/immutability controls, record external chain checkpoints, restrict DDL and privileged database access, and alert on missing exports. Those controls are deployment work and are not claimed by the local hash chain.

Avoid logging bearer tokens, signing keys, database passwords, raw credentials or unnecessary incident/log personal data. Apply retention/redaction policies to evidence, reports and audit details. Local container logs are diagnostic output, not durable audit retention.

Backup and restore

The PostgreSQL named volume protects against ordinary container replacement; it is not a backup. For a local demo, create a custom-format dump and copy it out of the database container:

```
docker compose exec postgres pg_dump -U sentinelops -d sentinelops -Fc -f
/tmp/sentinelops.dump
mkdir -p work/backups
docker compose cp postgres:/tmp/sentinelops.dump work/backups/sentinelops.dump
```

For a restore rehearsal, use a separate test database:

```
docker compose exec postgres createdb -U sentinelops sentinelops_restore
docker compose cp work/backups/sentinelops.dump postgres:/tmp/sentinelops-restore.dump
docker compose exec postgres pg_restore -U sentinelops -d sentinelops_restore --no-owner
/tmp/sentinelops-restore.dump
```

Do not point running production workers at a restored historical queue: old execution jobs can refer to actions already performed externally. Restore into an isolated environment with live tools disabled, inspect/reconcile job state and approvals, and plan the cutover before starting any worker.

In Cloud SQL, configure automated backups and point-in-time recovery according to the required recovery point/time. Test restoring into a different instance and verify incident, proposal and audit relationships. Store any exported dumps in restricted encrypted storage. Document who can restore and how external action state will be reconciled.

Release and rollback

1. Run locked dependency installation, security/workflow tests, fixture evaluations, TypeScript checks, frontend build and the full-stack smoke test.
2. Build immutable images. Record image digests, migration revision and measured fixture-report version. Use explicit Secret Manager versions.
3. Back up the database and inspect schema compatibility. Apply migration once with the migration identity before promoting API/worker revisions.
4. Deploy into staging, verify readiness and a role-controlled workflow, then promote the exact images/configuration.
5. For a code rollback, restore the previous image revision only if it can read the current schema. A container rollback does not undo schema changes or remote mutations. Prefer forward fixes/expand-contract migrations over destructive downgrades.

The GitHub workflow runs checks and artifacts; it does not deploy, hold cloud credentials or automatically promote changes to a production service.

Alerts and capacity decisions for production

Monitor oldest queued job, failed/ambiguous jobs, lease expiration, worker progress, API errors/latency, database connections/storage/CPU, denied execution spikes and audit-export lag. Define alert thresholds against the service's incident response requirements; this prototype has no measured production SLO to copy.

The initial Cloud Run limits are illustrative safety bounds, not load-test results. Account for API and worker connection pools when choosing database capacity. Use database/provider timeouts, maximum incident sizes and rate limits. Keep model token/cost budgets explicit before enabling model calls. Redis can later cache safe read results or hold rate-limit counters, but it is not currently on the correctness path and should never be the sole approval record.

Keep the worker's minimum replica and always-allocated CPU configuration in place while it polls the database. Reducing minimum replicas to zero may stop all work while API requests still appear successful.

Validation record

Recorded on 6 October 2026. This document reports executed checks and their limits, not a production certification.

Final application checks

Check	Observed result	What it establishes
Backend suite: <code>python -m pytest tests apps/api/tests -q</code>	97 passed , final owner run 2.56 seconds	Policy, API lifecycle, worker recovery and UTC serialization regressions on temporary SQLite databases.
Frontend TypeScript	Passed	Frontend source satisfies its configured TypeScript checks.
Frontend production build	Passed with Next.js 16.3.8	Static/application compilation and standalone build succeed.
Frontend dependency audit	0 reported vulnerabilities	npm's advisory database reported none for the final lockfile at the time of checking. This is not a universal security guarantee.
Ruff on backend and tests	Passed	Configured lint/style rules pass.
Fixture evaluator	64 cases, 0 errors	Real workflow/retrieval/policy code runs on explicit synthetic observations.
Native HTTP smoke	Passed	API and a separate worker complete the approval-to-resolution lifecycle with durable SQLite jobs.
Browser journey	Passed, no page errors observed	Operator creation/investigation, approver decision/execution, resolution, report, runbooks, policy, measured evals and architecture work through the UI.
Mobile browser	Passed at 390 × 844 viewport, no document horizontal overflow	Main console, new incident form, navigation and uncertain-evidence escalation remain usable.
Browser timezone	Passed in Asia/Kolkata	API UTC timestamps represent the correct instant rather than appearing five and a half hours old.

Check	Observed result	What it establishes
MCP stdio	Handshake, list/read succeeded; rollback denied	The real MCP protocol exposes typed fixture reads and blocks mutation bypass.
Python wheel	Built with migration assets	Packaged installation retains Alembic configuration and versions.

The UTC regression matters: SQLite drops timezone metadata when reading datetimes. The API now attaches UTC to its naive database timestamps and converts aware datetimes to UTC. Tests check actual JavaScript `Date.parse` behavior under Asia/Kolkata, as well as incident and audit serialization.

Real local container checks

The final Docker Compose stack was built and started in this workspace. PostgreSQL/pgvector, the migration task, the nonroot API, the supervised worker and the Next.js frontend passed their health checks. Validation used ports 8100/3100 to coexist with native browser tests; normal user defaults remain 8000/3000.

The HTTP smoke created a PostgreSQL-backed incident, queued investigation, observed a supported rollback proposal, rejected approval from the operator role, approved through the approver role, explicitly executed the simulation, verified resolution and replayed the same idempotency key safely.

The container evaluator loaded its default dataset and completed 64 fixture cases without errors. The API could read the mounted measured evaluation report. PostgreSQL had its vector extension installed.

Direct PostgreSQL audit guard checks attempted **UPDATE**, **DELETE** and **TRUNCATE** within rolled-back transactions. All three were rejected. The application ORM also blocks audit updates/deletes. Database administrators can still drop defenses or rewrite history; stronger retention requires external anchoring or immutable storage.

Cloud Run templates and their renderer were parsed/rendered locally. They have not been server-validated, applied or deployed to a cloud account. A final API container rebuild includes the UTC serialization patch.

Evaluation interpretation

The checked-in generated report is `evals/results/latest.json`. Its deterministic fixture baseline produced 1.0 for measured correctness and safety checks. Those results are intentionally labeled **synthetic deterministic integration results**. The classifier and fixture suite were designed together; this is not an independent held-out model benchmark.

The recorded report's local latency P50 is 2.298 ms and P95 is 7.825 ms. Container/native runs can vary with host load. These durations do not include live provider calls, customer integrations or production queues. Retrieval Recall@5 is measured over only four curated runbooks, so it does not establish semantic retrieval quality at enterprise scale.

Remediation success rate and average LLM cost remain null in the evaluation report because the harness stops at the approval gate and does not collect provider billing. Separate lifecycle tests execute simulated remediation. No actual system rollback or restart occurred.

Recorded evidence

- Investigation trace (included in the repository under docs/media/)
- Approved proposal (included in the repository under docs/media/)
- Resolved incident (included in the repository under docs/media/)
- Incident report (included in the repository under docs/media/)
- Measured evaluation dashboard (included in the repository under docs/media/)
- Mobile console (included in the repository under docs/media/)
- Incident workflow recording (included in the repository under docs/media/)

These are captures from the running native demo, using SQLite and synthetic telemetry. They are separate from the real PostgreSQL container validation above.

What remains unverified or intentionally unfinished

No OpenAI credential was configured, so the optional live-model path was not called. No enterprise provider, production mutation adapter, OAuth/OIDC login flow, tenant isolation, semantic embedding ingestion/search, OTLP export/context propagation, load capacity, managed backup restore, cloud IAM deployment or production recovery was tested.

The worker has bounded leases and protects against ambiguous mutation replay. Long-running future adapters require lease renewal or a reviewed longer-bound execution design. The current fixture mutation and database receipt cannot establish exactly-once behavior for a remote provider.

See [security.md](#), [deployment.md](#), [operations.md](#) and the production requirements in the README before configuring real infrastructure.

ADR 0001: use one explicit workflow orchestrator

Status: accepted for the initial implementation.

The incident lifecycle must expose evidence gathering, policy evaluation, waiting for approval and verification. We use LangGraph for the investigation graph and the OpenAI Responses API for optional structured reasoning. We do not introduce multiple overlapping agent orchestrators.

This gives the workflow one state and transition model. Application database jobs own durability at job boundaries. LangGraph persistence is not assumed merely because a graph is used. A future checkpoint design must explicitly address node replay, side effects and approval freshness.

Consequence: adding a second orchestration SDK requires a concrete capability and recovery argument. An experimental managed-agent branch can be compared with the same evals without changing the main safety boundary.

ADR 0002: database-backed work requests

Status: accepted for the initial implementation.

An investigation must survive API restarts and a human approval may arrive much later. We persist jobs alongside incident state in PostgreSQL and run a separate worker. Redis remains optional cache infrastructure.

Writing the incident transition and work request transactionally avoids the early dual-write problem of writing a database row and separately publishing to a broker. It also reduces local infrastructure requirements.

Consequence: monitor database contention, connection pools, job queue age and leases. SQLite is a single-worker learning fallback. At higher scale, a managed queue may be introduced with a transactional outbox and consumers that tolerate duplicates. Mutating work is not blindly replayed after an ambiguous worker failure.

ADR 0003: approval authorizes an exact action

Status: accepted for the initial implementation.

The agent may recommend a remediation but cannot authorize it. Rollback and restart proposals require an approver role. Approval stores the exact canonical proposal fingerprint. Execution rechecks identity, state, typed arguments, fingerprint and idempotency, independently of the model and UI.

Write SQL, destructive operations and unknown tools are denied. Database diagnostic tools accept named query templates rather than arbitrary model-produced SQL. MCP exposes read-only tools in the initial server; it does not create a second mutation route.

Consequence: real integrations must preserve this boundary and use scoped credentials. A changed proposal requires new approval. Production should add approval expiry, service/environment entitlements and configurable separation of duties.

ADR 0004: reproducible simulation before live infrastructure

Status: accepted for the initial implementation.

No enterprise metrics, deployments, logs or cloud credentials are configured in this workspace. The initial adapters simulate observations and state changes. This lets reviewers run the complete workflow and test dangerous-action rejection without operating real infrastructure.

Optional OpenAI reasoning can use the same observations, but it does not enable live tools. Evaluations report the execution mode and disclose synthetic data. Unknown and database-saturation cases may require more investigation instead of forcing a rollback or restart.

Consequence: this is a runnable production-oriented foundation, not a certified production incident responder. Every live adapter requires authorization, contract tests, staging failure exercises and operational ownership before activation.